INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS

Las Americas Institute of Technology

SUSTENTANTE:

Abel Eduardo Martínez Robles

MATRICULA:

2024-0227

MATERIA:

Programación Paralela

Trabajo Final:

Resolución de rompecabezas (Sudoku) usando paralelismo con Descomposición exploratoria

MAESTRO:

Erick Leonardo Perez Veloz

PERIODO ACADÉMICO:

2026-C1

Santo Domingo Norte, Rep. Dom

25 de Abril del 2026

Índice

Índice **1**
1. Introducción **2**
Objetivo general 2
Objetivos específicos 2
2. Descripción del problema **3**
Contexto del problema seleccionado 3
Aplicación del problema en un escenario real 3
Importancia del paralelismo en la solución 3
3. Cumplimiento de los Requisitos del Proyecto **4**
4. Diseño de la Solución **5**
Arquitectura general del sistema 5
Diagrama de componentes/tareas paralelas 6
Estrategia de paralelización utilizada 7
Herramientas y tecnologías empleadas (C#, TPL, etc.) 8
5. Implementación Técnica **9**
Descripción de la estructura del proyecto 9
Explicación del código clave 10
Uso de mecanismos de sincronización 11
Justificación técnica de las decisiones tomadas 11
6. Evaluación de Desempeño **12**
Comparativa entre ejecución secuencial y paralela 12
Métricas: tiempo de ejecución, eficiencia, escalabilidad 12
Gráficas o tablas con resultados 13
Análisis de cuellos de botella o limitaciones 14
7. Trabajo en Equipo **14**
8. Conclusiones **15**
9. Referencias **16**
10. Anexos **17**
Capturas adicionales y pruebas complementarias 18
Enlace al repositorio de Git: **20**

1. Introducción

Este proyecto consiste en el desarrollo de un sistema para la resolución de un Sudoku utilizando técnicas de programación paralela en C#. El objetivo principal es comparar el rendimiento entre una solución secuencial tradicional y una versión paralela, en la cual se evalúan múltiples posibles soluciones al mismo tiempo. Para esto, se aplica el enfoque de **descomposición exploratoria**, donde el problema se divide en diferentes caminos que pueden ser explorados de manera concurrente.

La elección de este tema se debe a que el Sudoku es un problema clásico de búsqueda y optimización que se adapta muy bien al uso del paralelismo. A medida que aumenta la dificultad del tablero, también crece el número de combinaciones posibles, lo que lo convierte en un buen candidato para demostrar cómo el paralelismo puede mejorar el rendimiento. Además, permite trabajar aspectos importantes como la sincronización entre tareas, el uso eficiente de los recursos del sistema y la evaluación del desempeño.

Objetivo general

Desarrollar un sistema en C# que resuelva un Sudoku utilizando programación paralela, con el fin de analizar y comparar su rendimiento frente a una solución secuencial.

Objetivos específicos

- Implementar un algoritmo secuencial para la resolución de Sudoku.
- Desarrollar una versión paralela que explore múltiples soluciones de forma simultánea.
- Analizar el comportamiento del sistema en diferentes niveles de dificultad.
- Medir el rendimiento utilizando métricas como tiempo de ejecución y speedup.
- Evaluar las ventajas y limitaciones del paralelismo en este tipo de problemas.

2. Descripción del problema

Contexto del problema seleccionado

El problema abordado en este proyecto es la resolución de un Sudoku, el cual consiste en completar una matriz de 9x9 con números del 1 al 9, cumpliendo ciertas reglas: no repetir números en filas, columnas ni en subcuadrículas de 3x3. Aunque puede parecer sencillo en niveles básicos, a medida que aumenta la dificultad, el número de combinaciones posibles crece significativamente, convirtiéndose en un problema de búsqueda con muchas posibles soluciones intermedias.

A medida que el nivel de dificultad del Sudoku aumenta, también lo hace la cantidad de posibles combinaciones que deben analizarse. Esto provoca que el proceso de resolución pueda volverse más costoso en términos de tiempo si se realiza de forma secuencial, ya que el algoritmo debe explorar muchas posibilidades antes de llegar a la solución correcta.

Aplicación del problema en un escenario real

Aunque el Sudoku es un juego, su lógica representa problemas reales donde es necesario evaluar múltiples alternativas para encontrar una solución óptima o válida. Este tipo de enfoque se utiliza en áreas como la planificación de tareas, la optimización de recursos, la inteligencia artificial y la resolución de problemas complejos en ingeniería.

Por ejemplo, en sistemas de asignación de recursos, se deben probar distintas combinaciones para determinar la mejor distribución posible sin violar ciertas restricciones. De forma similar, en inteligencia artificial, muchos algoritmos deben explorar diferentes estados o caminos antes de encontrar una solución adecuada. El Sudoku sirve como un modelo simplificado para representar este tipo de problemas.

Importancia del paralelismo en la solución

El uso del paralelismo en este problema permite mejorar el proceso de búsqueda al evaluar múltiples posibles soluciones de manera simultánea. En lugar de analizar cada opción de forma secuencial, el problema se divide en varios caminos que pueden ser explorados al mismo tiempo utilizando múltiples tareas.

Esto resulta especialmente útil en Sudokus de mayor dificultad, donde el número de combinaciones posibles es mucho mayor. Sin embargo, el paralelismo también introduce desafíos, como la necesidad de sincronizar los datos compartidos entre tareas y evitar conflictos cuando varias tareas intentan acceder o modificar la misma información.

Por esta razón, este problema permite no solo demostrar las ventajas del paralelismo en términos de rendimiento, sino también analizar sus limitaciones y la importancia de una correcta gestión de la concurrencia.

3. Cumplimiento de los Requisitos del Proyecto

- **Ejecución simultánea de múltiples tareas**

El proyecto implementa la ejecución simultánea de múltiples tareas mediante el uso de *Task* en C#. En la versión paralela del algoritmo, cuando se encuentra una celda vacía en el Sudoku, se generan varias tareas que prueban diferentes valores posibles al mismo tiempo. Cada tarea explora un camino distinto de solución, lo que permite aprovechar varios núcleos del procesador y reducir el tiempo de búsqueda en comparación con la ejecución secuencial.

- **Necesidad de compartir datos entre tareas**

Durante la ejecución paralela, las tareas comparten información importante como el estado de si ya se encontró una solución (*solutionFound*) y el tablero solución (*solutionBoard*). Para evitar conflictos entre tareas al acceder o modificar estos datos, se utiliza sincronización mediante *lock*, asegurando que solo una tarea pueda escribir la solución final. Esto permite mantener la consistencia de los datos en un entorno concurrente.

- **Exploración de diferentes estrategias de paralelización**

El proyecto permite explorar distintas formas de aplicar el paralelismo. Actualmente, se utiliza una estrategia basada en dividir el problema en múltiples caminos desde la primera celda vacía y asignar cada uno a una tarea. Además, se controla la cantidad de tareas concurrentes utilizando *SemaphoreSlim*, lo que evita sobrecargar el sistema. Este enfoque puede ser modificado o ampliado para probar otras estrategias, como limitar la profundidad del paralelismo o combinar ejecución secuencial y paralela.

- **Escalabilidad con más recursos**

El sistema tiene la capacidad de escalar con los recursos disponibles del equipo. Esto se logra utilizando ***Environment.ProcessorCount*** para determinar la cantidad de núcleos del procesador y ajustar el número de tareas concurrentes. De esta manera, en equipos con más núcleos, el programa puede ejecutar más tareas en paralelo, mejorando el rendimiento en problemas más complejos.

- **Métricas de evaluación del rendimiento**

El proyecto incluye métricas para evaluar el rendimiento del sistema. Se mide el tiempo de ejecución tanto en la versión secuencial como en la paralela utilizando *Stopwatch*. Además, se calcula el speedup para comparar ambas ejecuciones y se exportan los resultados automáticamente en archivos *.txt* y *.csv*, lo que permite analizar los datos y generar gráficas para visualizar el comportamiento del sistema.

- **Aplicación a un problema del mundo real**

Aunque el Sudoku es un juego, su resolución representa un problema de búsqueda y optimización que se aplica en escenarios reales. Este tipo de enfoque se utiliza en áreas como la inteligencia artificial, la planificación de tareas y la optimización de recursos, donde es necesario evaluar múltiples combinaciones para encontrar una solución válida. El proyecto demuestra cómo el paralelismo puede ser aplicado en este tipo de problemas para mejorar el rendimiento.

4. Diseño de la Solución

Arquitectura general del sistema

El sistema sigue una arquitectura modular en la que cada componente cumple una función específica dentro del proceso de resolución del Sudoku.

El flujo de ejecución comienza en el módulo principal (**Program**), el cual se encarga de cargar los tableros, ejecutar las versiones secuencial y paralela del algoritmo y gestionar la medición de rendimiento.

Existen dos enfoques principales para resolver el problema:

- Un enfoque **secuencial**, donde el tablero se resuelve utilizando backtracking de forma recursiva.
- Un enfoque **paralelo**, donde el problema se divide en múltiples tareas que exploran diferentes posibles soluciones al mismo tiempo.

Ambos enfoques utilizan un módulo común encargado de validar las reglas del Sudoku, lo que permite reutilizar la lógica del problema.

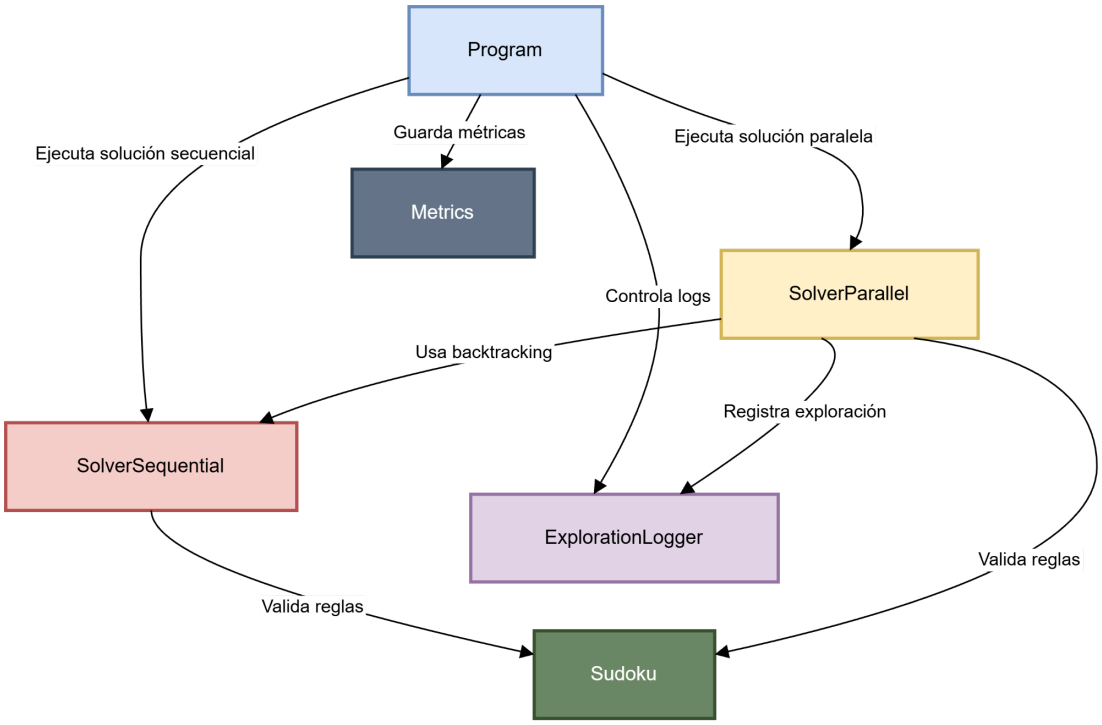
Durante la ejecución paralela, múltiples tareas trabajan simultáneamente sobre diferentes caminos de solución, compartiendo información sobre el estado global (por ejemplo, si ya

se encontró una solución). Para garantizar la correcta ejecución, se aplican mecanismos de sincronización.

Finalmente, el sistema incluye módulos adicionales para registrar el comportamiento de la ejecución paralela y almacenar métricas de rendimiento, lo que permite comparar ambos enfoques.

Esta arquitectura permite separar claramente la lógica del problema, la ejecución paralela y la evaluación del rendimiento.

Diagrama de componentes/tareas paralelas

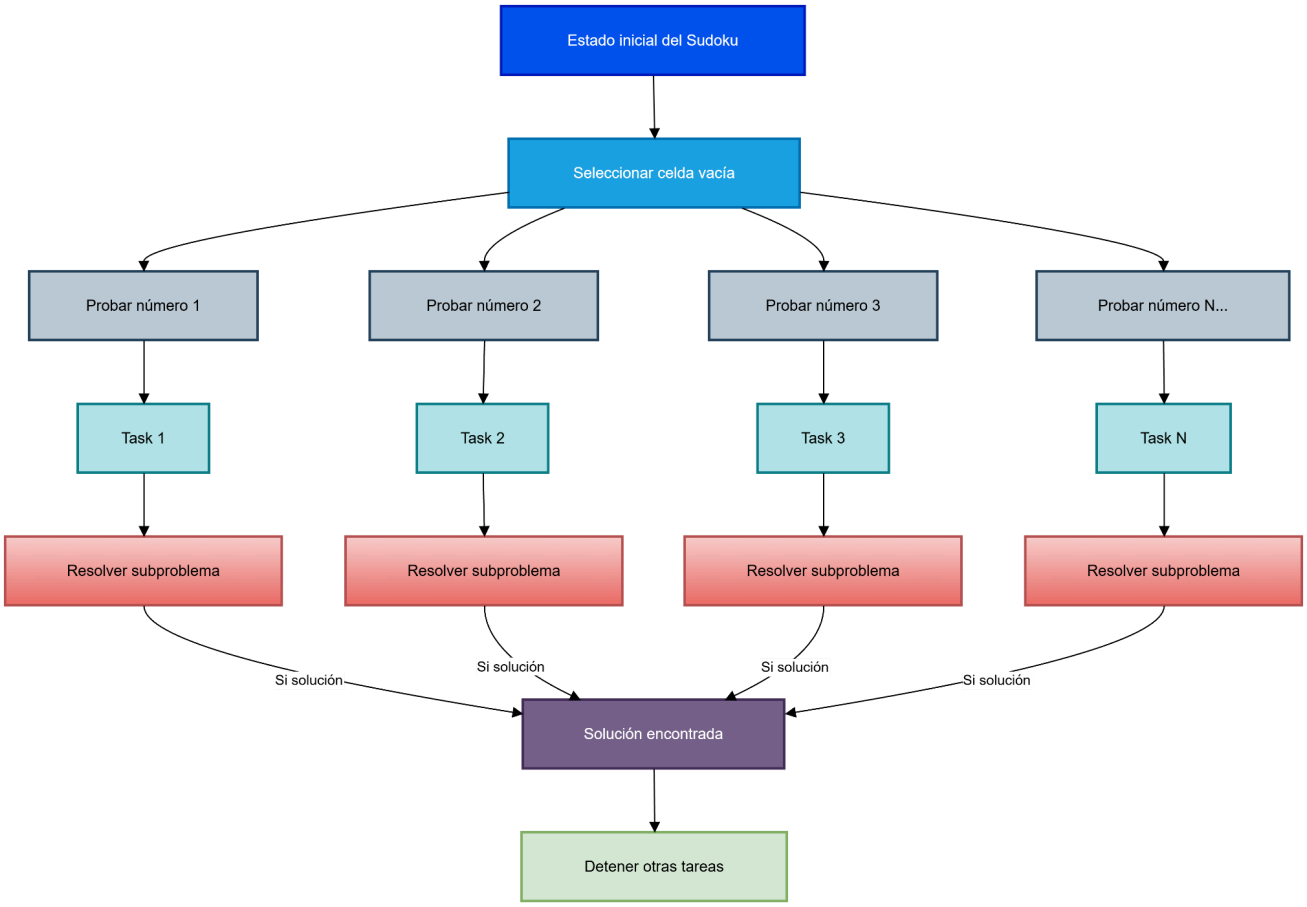


El sistema está compuesto por varios módulos donde **Program** controla la ejecución.

SolverSequential resuelve el Sudoku de forma tradicional, mientras que **SolverParallel** ejecuta varias tareas al mismo tiempo para explorar diferentes soluciones.

Ambos utilizan **Sudoku** para validar el tablero.

Además, **Metrics** guarda los tiempos de ejecución y **ExplorationLogger** registra cómo trabajan las tareas en paralelo.



El problema se divide a partir de una celda vacía, generando varios posibles valores.

Cada valor se convierte en una tarea que se ejecuta en paralelo, explorando diferentes caminos al mismo tiempo.

Cuando una tarea encuentra la solución, se detienen las demás.

Esto demuestra la **descomposición exploratoria**, ya que el problema se divide en múltiples posibles soluciones.

Estrategia de paralelización utilizada

El proyecto utiliza una estrategia de **descomposición exploratoria**, donde el problema se divide en múltiples posibles caminos de solución.

En la implementación, cuando se encuentra una celda vacía en el tablero, se generan diferentes tareas, cada una probando un valor distinto en esa posición. Cada tarea trabaja con una copia del tablero y ejecuta el algoritmo secuencial de forma independiente.

Para controlar la ejecución, se utilizan los siguientes mecanismos:

- **Creación de tareas (*Task*)**: Permite ejecutar múltiples caminos en paralelo.
- ***SemaphoreSlim***: Limita la cantidad de tareas concurrentes según los núcleos disponibles, evitando sobrecargar el sistema.
- ***lock***: Se utiliza para asegurar que solo una tarea pueda escribir la solución final, evitando conflictos de concurrencia.

Este enfoque permite explorar varias soluciones al mismo tiempo, mejorando el rendimiento en problemas más complejos, aunque también introduce cierta sobrecarga en casos simples.

Herramientas y tecnologías empleadas (C#, TPL, etc.)

Para el desarrollo del proyecto se utilizaron las siguientes herramientas y tecnologías:

- **C#**: Lenguaje principal utilizado para la implementación del sistema.
- **.NET**: Plataforma sobre la cual se ejecuta la aplicación.
- **Task Parallel Library (TPL)**: Utilizada para la creación y gestión de tareas en paralelo (*Task, Task.Run*).
- **SemaphoreSlim**: Herramienta para controlar la concurrencia y limitar la cantidad de tareas activas.
- **lock (sincronización)**: Mecanismo para proteger el acceso a recursos compartidos.
- **Stopwatch**: Utilizado para medir el tiempo de ejecución en ambas versiones (secuencial y paralela).
- **Manejo de archivos (System.IO)**: Para guardar resultados en archivos *.txt* y *.csv.*

Estas herramientas permiten implementar y evaluar de manera efectiva el paralelismo dentro del sistema.

5. Implementación Técnica

Descripción de la estructura del proyecto

El proyecto está organizado en diferentes carpetas y módulos con el objetivo de separar responsabilidades y facilitar el mantenimiento del sistema.

Estructura del repositorio:

- **/docs**: Contiene la documentación del proyecto, incluyendo la explicación del problema, diseño de la solución y análisis de resultados.
- **/src**: Contiene el código fuente del sistema.
- **/tests**: Destinada para pruebas del sistema (puede incluir casos de prueba adicionales).
- **/metrics**: Almacena los resultados generados durante la ejecución, como archivos *.txt, .csv* y logs de exploración.

Dentro de la carpeta **/src**, el proyecto sigue la estructura generada por Visual Studio:

```
/src
 └── FinalProject        ← Solución (archivo .sln)
     └── FinalProject    ← Proyecto principal (código fuente)
```

Esta duplicación de nombres se debe a que Visual Studio crea una carpeta para la solución y otra para el proyecto. Dentro del proyecto principal se encuentran los archivos desarrollados:

```
 ├── Program.cs
 ├── Sudoku.cs
 ├── SolverSequential.cs
 ├── SolverParallel.cs
 ├── Metrics.cs
 └── ExplorationLogger.cs
```

- **Program.cs**: Es el punto de entrada del sistema. Se encarga de definir los casos de prueba (Sudokus), ejecutar las versiones secuencial y paralela, y mostrar los resultados en consola. También coordina la medición de tiempos y el guardado de métricas.

- **Sudoku.cs**: Contiene las funciones básicas relacionadas con el tablero, como la validación de movimientos (*IsValid*), la búsqueda de celdas vacías (*FindEmpty*) y la impresión del tablero. Este componente centraliza la lógica del problema.
- **SolverSequential.cs**: Implementa la solución tradicional mediante backtracking. Explora el tablero de forma recursiva, probando valores uno por uno hasta encontrar una solución válida.
- **SolverParallel.cs**: Implementa la versión paralela del algoritmo. Divide el problema en múltiples tareas que exploran diferentes caminos de solución al mismo tiempo. También maneja la sincronización de datos compartidos y el control de concurrencia.
- **Metrics.cs**: Se encarga de medir y almacenar los resultados de rendimiento, incluyendo tiempos de ejecución y speedup, guardándolos en archivos para su posterior análisis.
- **ExplorationLogger.cs**: Registra el comportamiento de las tareas paralelas, mostrando qué caminos se están explorando y cuál encuentra la solución. Esto permite entender cómo se ejecuta el algoritmo internamente.

Explicación del código clave

El funcionamiento del sistema se basa en dos enfoques principales:

- **Algoritmo secuencial (SolverSequential)**
  Utiliza backtracking para recorrer el tablero de forma recursiva, probando valores del 1 al 9 en cada celda vacía hasta encontrar una solución válida.
- **Algoritmo paralelo (SolverParallel)**
  Aplica descomposición exploratoria, generando múltiples tareas a partir de una celda vacía.
  Cada tarea explora una posible solución de manera independiente, reutilizando el algoritmo secuencial.

Además, el sistema incluye:

- Medición de tiempo con *Stopwatch*
- Registro de resultados en archivos
- Logs de exploración para visualizar el comportamiento paralelo

Uso de mecanismos de sincronización

El sistema utiliza diferentes mecanismos para manejar la concurrencia:

- **Variable compartida (*solutionFound*)**
  Permite indicar cuándo una tarea ha encontrado la solución.
- **Bloque** *lock*
  Se utiliza para evitar condiciones de carrera al guardar la solución final.
- ***SemaphoreSlim***
  Controla la cantidad de tareas activas, limitándolas según los núcleos del procesador para evitar sobrecarga.

Estos mecanismos aseguran que las tareas paralelas trabajen de forma segura y eficiente.

Justificación técnica de las decisiones tomadas

Se tomaron varias decisiones técnicas para garantizar un buen desempeño del sistema:

- Se utilizó **backtracking** por ser una técnica adecuada para problemas de búsqueda como el Sudoku.
- Se aplicó **paralelismo a nivel de exploración**, ya que el problema permite dividirse en múltiples caminos independientes.
- Se usó **SemaphoreSlim** para evitar crear demasiadas tareas y afectar el rendimiento.
- Se reutilizó el solver secuencial dentro del paralelo para mantener simplicidad y evitar duplicación de lógica.
- Se implementó un sistema de **métricas automatizado** para comparar el rendimiento entre ambas versiones.

Estas decisiones permiten demostrar claramente las ventajas y limitaciones del paralelismo en este tipo de problemas.

6. Evaluación de Desempeño

Comparativa entre ejecución secuencial y paralela

Se realizaron pruebas con tres niveles de dificultad: *fácil, medio y difícil,* con el objetivo de comparar el comportamiento del algoritmo secuencial frente al paralelo.

Los resultados muestran que el paralelismo no siempre mejora el rendimiento. En el caso fácil, el tiempo paralelo es mayor debido al costo de crear y gestionar tareas. En el caso medio, el paralelismo demuestra su ventaja al reducir significativamente el tiempo de ejecución. En el caso difícil, el tiempo secuencial es muy bajo, por lo que el paralelismo introduce una sobrecarga innecesaria.

Métricas: tiempo de ejecución, eficiencia, escalabilidad

Se utilizaron las siguientes métricas:

- **Tiempo de ejecución**: medido en milisegundos para ambas versiones.
- **Speedup**: relación entre el tiempo secuencial y el paralelo.
- **Eficiencia**: indica qué tan bien se aprovechan los recursos paralelos.

Caso	Secuencial (ms)	Paralelo (ms)	Speedup
Sudoku Fácil	4.56	7.2967	0.62x
Sudoku Medio	697.3627	357.0434	1.95x
Sudoku Difícil	0.1378	2.2739	0.06x

Interpretación:

- **Speedup > 1** → el paralelismo mejora el rendimiento
- **Speedup < 1** → el paralelismo empeora el rendimiento

En este caso, el mejor resultado se obtiene en el Sudoku medio, donde el problema tiene suficiente complejidad para aprovechar el paralelismo.

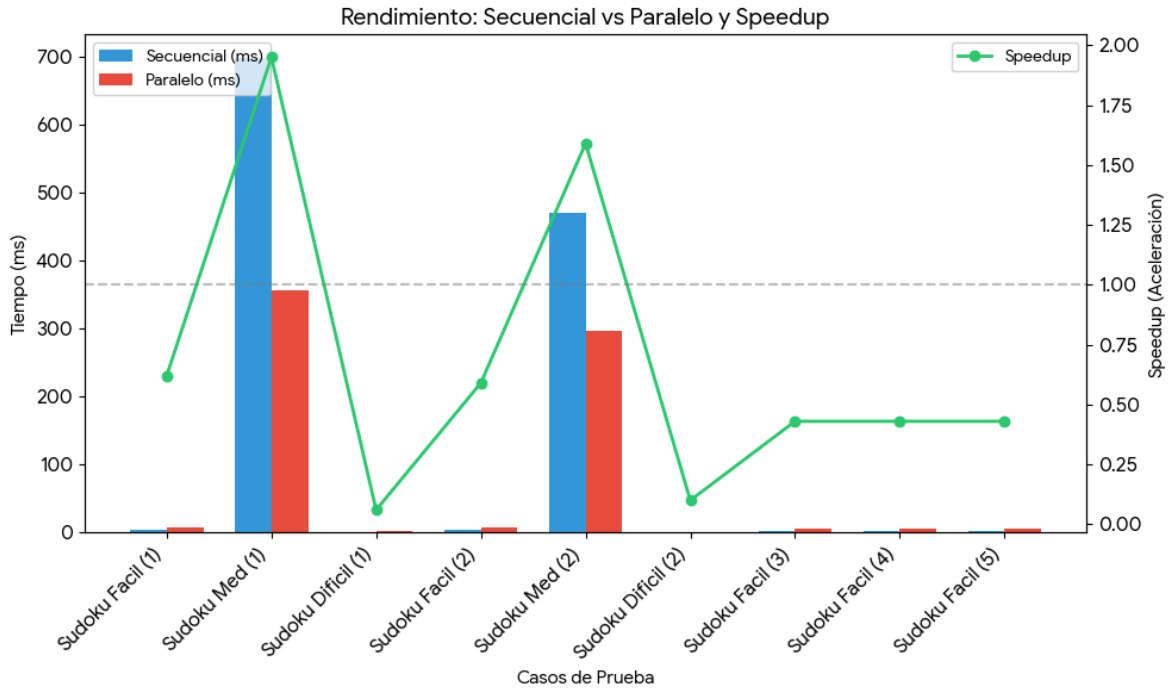
Gráficas o tablas con resultados

Los resultados fueron almacenados automáticamente en archivos CSV, lo que permite generar gráficas en herramientas como Excel.

A partir de estos datos, se pueden crear gráficos comparativos donde se observe:

- Tiempo secuencial vs paralelo por caso
- Speedup por nivel de dificultad

Estas gráficas permiten visualizar claramente que el paralelismo es más eficiente en problemas de mayor complejidad.



Análisis del Gráfico:

Barras Azules y Rojas: Representan el tiempo en milisegundos. Como podemos ver, en los casos de Sudoku Med, el tiempo secuencial (azul) es muy superior al paralelo (rojo), lo que demuestra que el procesamiento en paralelo es efectivo para problemas más complejos.

Línea Verde (Speedup): Muestra el factor de aceleración.

Cualquier punto por encima de la línea punteada (1.0) indica que la versión paralela es más rápida. Esto ocurre claramente en los casos "Med".

En los casos "Fácil" y "Difícil", el speedup es menor a 1, lo que significa que el programa tardó más en paralelo. Esto suele pasar cuando el trabajo es tan pequeño que se tarda más tiempo coordinando los hilos que procesando los datos.

Análisis de cuellos de botella o limitaciones

El análisis del rendimiento muestra que el paralelismo no siempre mejora el tiempo de ejecución. En los casos de Sudoku medio, donde el problema requiere una mayor exploración de soluciones, la versión paralela logra un speedup mayor a 1, demostrando una mejora significativa en el rendimiento. Esto se debe a que múltiples tareas trabajan simultáneamente en distintas ramas del problema, reduciendo el tiempo total.

Sin embargo, en los casos fácil y difícil, el speedup es menor a 1. Esto indica que la versión paralela es más lenta debido a la sobrecarga de crear y sincronizar tareas (threads), lo que se convierte en un cuello de botella. En estos casos, el problema es demasiado simple o se resuelve muy rápido, por lo que el costo del paralelismo supera sus beneficios.

En conclusión, el principal cuello de botella identificado es la gestión de concurrencia (creación de tareas, sincronización y uso de recursos compartidos). El paralelismo resulta más eficiente únicamente cuando el problema tiene suficiente complejidad para justificar su uso.

7. Trabajo en Equipo

Descripción del reparto de tareas

El proyecto fue desarrollado de manera individual, por lo que todas las responsabilidades fueron asumidas por una sola persona. Las tareas se organizaron por fases para mantener un flujo de trabajo ordenado:

- Análisis del problema y definición del enfoque de paralelismo
- Implementación de la solución secuencial
- Desarrollo de la versión paralela con manejo de concurrencia
- Integración de métricas de rendimiento
- Registro de la exploración de tareas
- Pruebas con distintos niveles de dificultad (fácil, medio, difícil)

- Análisis de resultados y documentación final

Esta organización permitió avanzar de forma progresiva, validando cada parte antes de continuar con la siguiente.

Herramientas utilizadas para coordinación
Aunque el proyecto fue individual, se utilizaron herramientas para mantener el control y la organización del desarrollo:

- **Git**: Para el control de versiones y seguimiento de cambios en el código.
- **Visual Studio**: Como entorno principal de desarrollo en C#.
- **Archivos CSV y TXT**: Para almacenar y analizar métricas de rendimiento.
- **Draw.io / Mermaid**: Para la creación de diagramas del sistema y tareas paralelas.
- **Excel**: Para generar gráficos y analizar los resultados obtenidos.

Estas herramientas ayudaron a mantener el proyecto organizado, facilitar el análisis de resultados y documentar correctamente el proceso.

8. Conclusiones

Principales aprendizajes técnicos
Durante el desarrollo del proyecto se logró comprender cómo aplicar el paralelismo a un problema de búsqueda como el Sudoku. Se aprendió a dividir el problema en múltiples tareas independientes (descomposición exploratoria) y a ejecutarlas de forma concurrente usando herramientas como TPL en C#. También se reforzó el uso de mecanismos de sincronización (locks, semáforos) para evitar conflictos entre tareas y garantizar resultados correctos. Además, se adquirió experiencia midiendo el rendimiento y comparando ejecuciones secuenciales vs paralelas mediante métricas como el tiempo y el speedup.

Retos enfrentados y superados
Uno de los principales retos fue manejar correctamente la concurrencia, especialmente al compartir datos como la solución encontrada. Al inicio, se presentaron problemas donde la solución paralela no se guardaba correctamente o generaba resultados vacíos, lo que obligó a implementar sincronización adecuada. Otro reto fue entender que el paralelismo no siempre mejora el rendimiento, ya que en problemas pequeños la sobrecarga de los hilos puede afectar negativamente. También fue un desafío estructurar el proyecto y organizar el código para que fuera claro y mantenible.

Posibles mejoras o líneas futuras

Como mejoras, se podría implementar una mejor estrategia de paralelización, por ejemplo, limitando la profundidad donde se crean tareas o aplicando técnicas híbridas más eficientes. También se podría agregar una interfaz gráfica para visualizar el proceso de resolución en tiempo real. Otra mejora sería ampliar el sistema de métricas, incluyendo uso de CPU y memoria, así como ejecutar pruebas con más casos para obtener resultados más robustos. Finalmente, este enfoque podría aplicarse a otros problemas más complejos de búsqueda y optimización, como algoritmos de inteligencia artificial o planificación de rutas.

9. Referencias

Microsoft. (s.f.). *Task Parallel Library (TPL)*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-parallel-library-tpl

Microsoft. (s.f.). *Parallel programming in .NET*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, & Clifford Stein. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Michael Vance, & Aleksei Fedorov. (2010). *Parallel programming with Microsoft .NET*. Microsoft Press.

Donald Knuth. (1997). *The art of computer programming* (Vol. 1–3). Addison-Wesley.

diagrams.net. (s.f.). *diagrams.net*. https://app.diagrams.net/

Mermaid. (s.f.). *Mermaid diagramming and charting tool*. https://mermaid.js.org/

10. Anexos

Manual de ejecución del sistema

Para ejecutar el proyecto, se deben seguir los siguientes pasos:

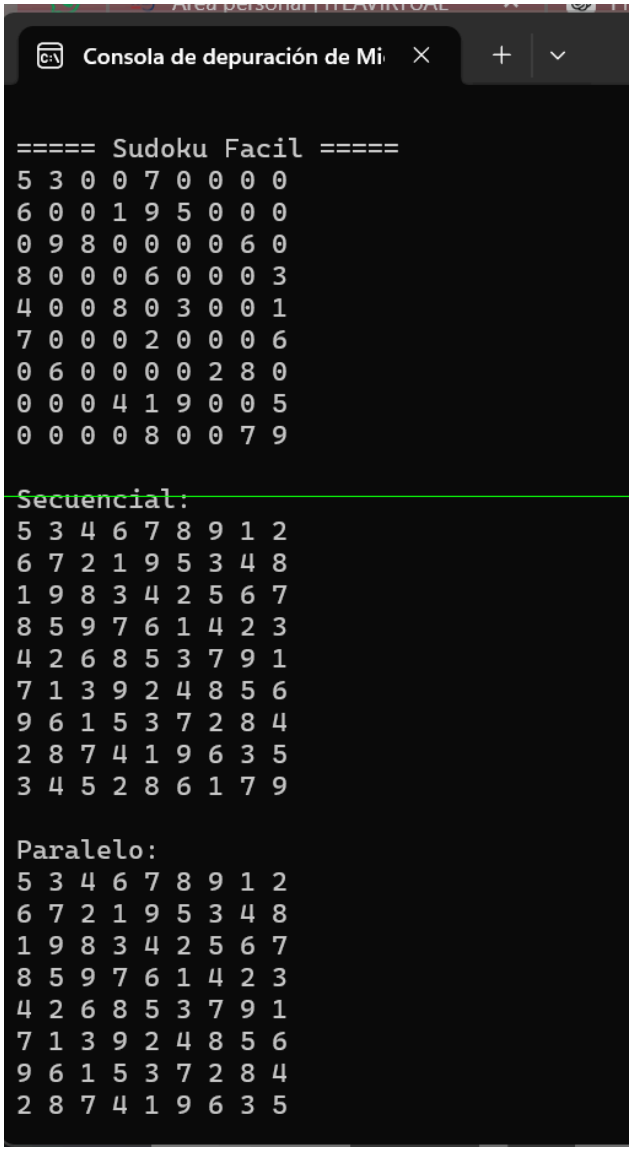
1. **Abrir el proyecto**
   - Abrir la solución en Microsoft Visual Studio o utilizar la terminal con el SDK de .NET instalado.
2. **Ubicación del proyecto**
   - El código fuente se encuentra en: */src/FinalProject/FinalProject*
3. **Compilar y ejecutar**
   - Desde Visual Studio: presionar **Run (F5)**.
   - Desde terminal, ubicarse en la carpeta del proyecto y ejecutar:

```
dotnet run
```

4. **Ejecución del programa**
   - El sistema resolverá automáticamente tres casos de Sudoku: fácil, medio y difícil.
   - Para cada caso se mostrará:
     - Tablero original
     - Solución secuencial
     - Solución paralela
     - Tiempos de ejecución
     - Speedup
5. **Resultados generados**
   - Los resultados se almacenan automáticamente en la carpeta */metrics,* incluyendo:
     - ***resultados.txt*** (resumen de ejecuciones)
     - ***resultados.csv*** (datos para gráficas)
     - Archivos de exploración por cada caso (****_exploracion.txt***)

Capturas adicionales y pruebas complementarias

- **Capturas de ejecución**
    - Se incluyen imágenes de la consola mostrando la ejecución del sistema para los distintos niveles de dificultad.



Se toma de referencia estos sudokus:

Fácil

5	3	.	.	.	7	.	.	.
6	.	.	1	9	5	.	.	.
.	9	8	.	.	.	.	6	.
8	.	.	.	6	.	.	.	3
4	.	.	8	.	3	.	.	1
7	.	.	.	2	.	.	.	6
.	6	.	.	.	.	2	8	.
.	.	.	4	1	9	.	.	5
.	.	.	.	8	.	.	7	9

Medio

.	.	.	6	.	.	4	.	.
7	.	.	.	.	3	6	.	.
.	.	.	.	9	1	.	8	.
.	.	.	.	.	.	.	.	.
.	5	.	1	8	.	.	.	3
.	.	.	3	.	6	.	4	5
.	4	.	2	.	.	.	6	.
9	.	3	.	.	.	.	.	.
.	2	.	.	.	.	1	.	.

Difícil

.	.	.	.	.	.	.	1	2
.	.	.	.	.	.	.	3	.
.	.	1	.	9	5	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	5	.	8	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	6	7	.	2	.	.
.	3	.	.	.	.	.	.	.
1	2	.	.	.	.	.	.	.

Se pueden ver los resultados y logs del programa

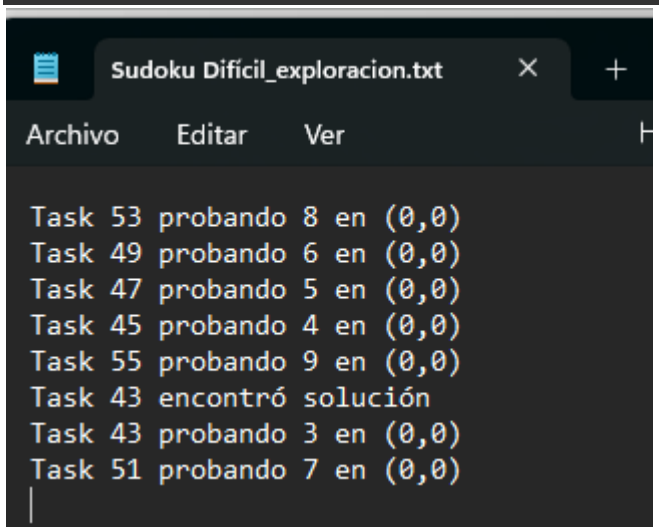
```
Caso: Sudoku Facil
Secuencial: 2.4296 ms
Paralelo: 5.6922 ms
Speedup: 0.43x
```

```
Caso: Sudoku Medio
Secuencial: 522.7359 ms
Paralelo: 305.8817 ms
Speedup: 1.71x
--------------------------------
```

```
Caso: Sudoku Dificil
Secuencial: 0.1311 ms
Paralelo: 0.9837 ms
Speedup: 0.13x
```

- **Gráficas de rendimiento**
    - Se generaron gráficas en Microsoft Excel a partir del archivo CSV, comparando:
        - Tiempo secuencial vs paralelo
        - Speedup por caso
- **Logs de exploración paralela**
    - Los archivos de exploración muestran cómo las tareas paralelas evalúan diferentes caminos de solución.
    - Ejemplo:

```
Task 41 probando 8 en (0,0)
Task 39 encontró solución
```



    - Esto evidencia la descomposición exploratoria del problema.
- **Pruebas realizadas**
    - Se ejecutaron múltiples veces los mismos casos para validar consistencia en los resultados.
    - Se compararon tiempos para analizar el impacto del paralelismo en distintos niveles de complejidad.

Enlace al repositorio de Git:

https://github.com/abelmartinez04/Proyecto-Final-Paralela